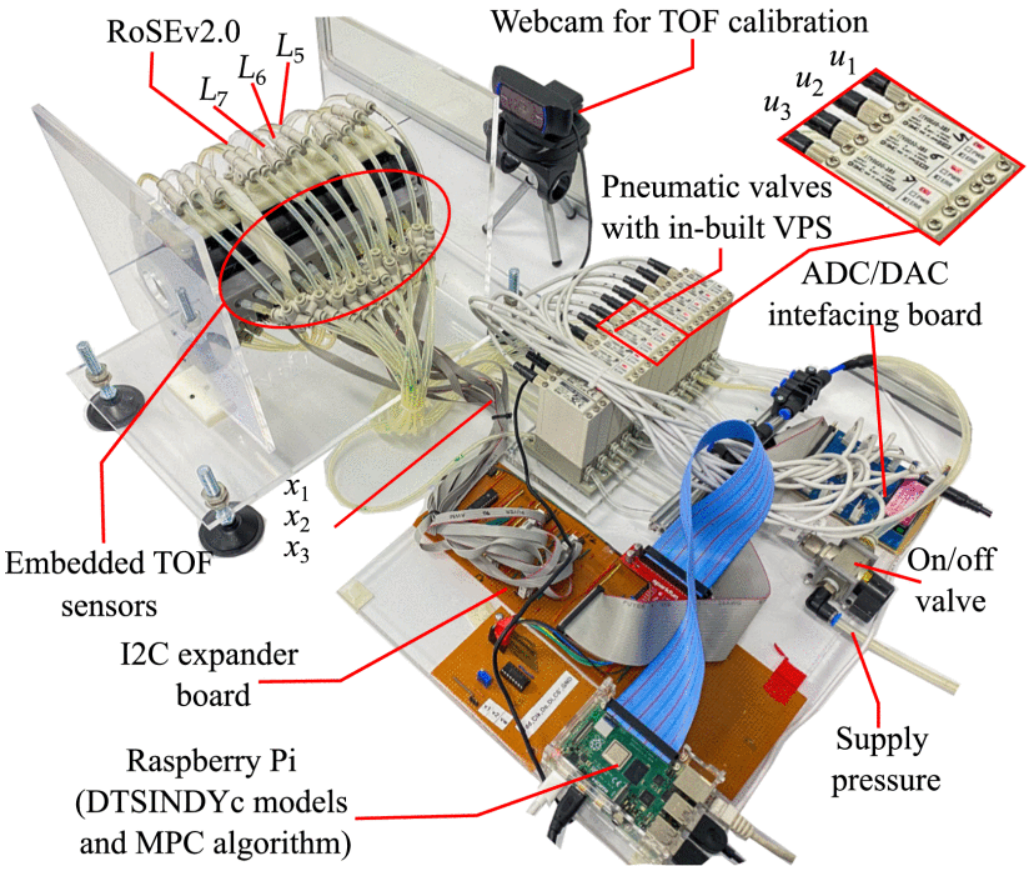


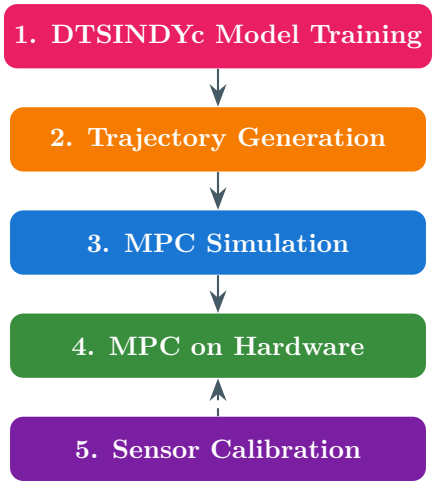
SINDYc-MPC for RoSE v2.0 Peristaltic Soft Robot

Comprehensive Technical Notes

Discrete-Time Sparse Identification of Nonlinear Dynamics with Control combined with Model Predictive Control

Dr Dipankar Bhattacharya

Imperial College London


1. DTSINDYc Model Training
2. Trajectory Generation
3. MPC Simulation
4. MPC on Hardware
5. Sensor Calibration

Contents

1 Introduction and System Overview **2**
1.1 Control Objective . . . 2

2 Hardware Architecture **2**
2.1 System Block Diagram . . . 2
2.2 VL6180X Time-of-Flight (TOF) Sensor . . . 3
2.2.1 TOF Calibration . . . 3
2.3 MCP3008 ADC Pressure Sensor . . . 4
2.4 Signal Conditioning — FIR Low-Pass Filter . . . 4

3 Stage 1 — DTSINDYc Model Training **5**
3.1 Mathematical Formulation . . . 5
3.1.1 State and Control Vectors . . . 5
3.1.2 Polynomial Library Construction . . . 6
3.1.3 Sparse Regression (STLSQ) . . . 7
3.1.4 PySINDy Model Fitting . . . 8

4 Stage 2 — Reference Trajectory Generation **8**
4.1 Mathematical Definition . . . 9
4.2 Phase-Shifted Three-Channel Wave . . . 9

5 Stage 3 — Model Predictive Control (MPC) **10**
5.1 MPC Cost Function . . . 10
5.2 Weight Matrices and Parameters . . . 10
5.3 State-Dependent Control Constraints . . . 11
5.4 MPC Block Diagram . . . 11

6 Stage 4 — Real-Time MPC on Hardware **12**
6.1 Control Loop Architecture . . . 12
6.2 3-State to 12-Layer Mapping . . . 12

7 Stage 5 — Sensor Calibration **13**
7.1 TOF Calibration Procedure . . . 13
7.2 ADC Calibration (Full-Scale Pressure) . . . 13

8 Data Flow and Directory Map **13**

9 Key Equations Summary **14**

10 Installation and Getting Started **14**
10.1 Environment Setup . . . 14
10.2 Raspberry Pi Additional Dependencies . . . 15
10.3 Running the Pipeline . . . 15
10.4 Repository File Organization . . . 15

11 Appendix: Complete Parameter Reference **16**

1 Introduction and System Overview

What is RoSE v2.0?

The **Robot of Soft Elastomer (RoSE) v2.0** is a 12-layer peristaltic soft robot fabricated from silicone elastomer (Ecoflex 00-50). Each layer contains a pneumatic chamber that can be independently inflated/deflated. By actuating layers in sequence with phase-shifted sinusoidal signals, the robot produces **peristaltic locomotion** — a traveling wave contraction pattern similar to biological organisms.
The robot is instrumented with two types of embedded sensors:

- **VL6180X Time-of-Flight (TOF)** sensors — measure layer displacement in mm
- **MCP3008 ADC / VPS** sensors — measure internal pneumatic pressure in mV

1.1 Control Objective

The goal is to track a desired peristaltic wave trajectory across three monitored layers (L5, L6, L7) using Model Predictive Control (MPC). The control pipeline is:

1. **Learn** a data-driven discrete-time dynamic model (DTSINDYc)
2. **Generate** a peristaltic wave reference trajectory
3. **Optimize** control inputs using MPC with the learned model
4. **Deploy** the controller on a Raspberry Pi in real time

2 Hardware Architecture

2.1 System Block Diagram

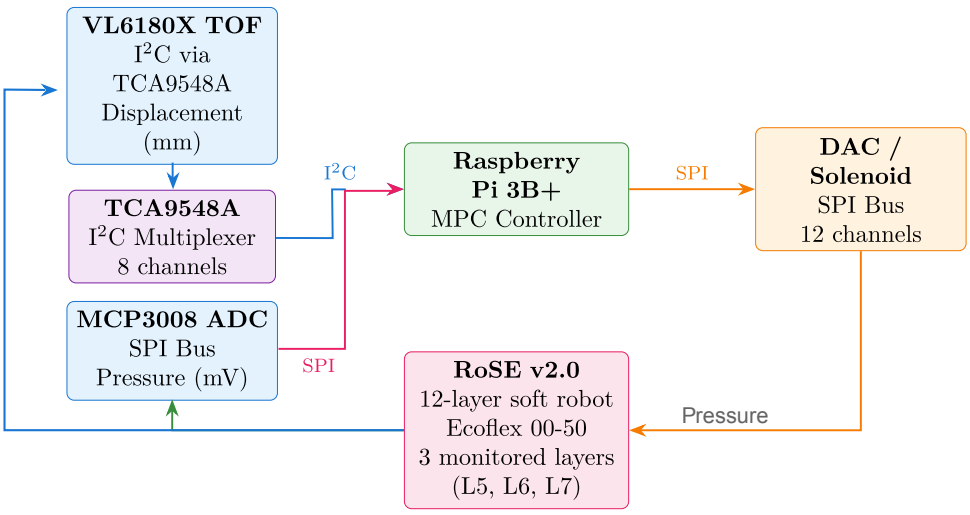


Figure 1: Hardware architecture of the RoSE v2.0 system. The Raspberry Pi interfaces with TOF sensors via an I²C multiplexer and reads ADC values for pressure sensing. Control signals are sent to the pneumatic actuators through a DAC.

2.2 VL6180X Time-of-Flight (TOF) Sensor

TOF Sensor Specifications

The **VL6180X** is a proximity/ambient light sensor from STMicroelectronics that uses a VCSEL (Vertical-Cavity Surface-Emitting Laser) infrared emitter and a SPAD (Single Photon Avalanche Diode) detector.

Parameter	Value
Measurement principle	Time-of-Flight (phase shift)
Range	0–200 mm (effective: 5–100 mm)
Resolution	∼1 mm
Interface	I^2C (default address: `0x29`)
Measurement rate	Up to 10 Hz (single-shot mode)
Supply voltage	2.8 V
Field of view	25° (cone)

Since all VL6180X sensors share the same I^2C address (`0x29`), a **TCA9548A** I^2C multiplexer is used to address each sensor on a separate channel. The multiplexer supports 8 channels, of which 6 are used for the 6 TOF sensors embedded in the robot.

2.2.1 TOF Calibration

Each TOF sensor requires three calibration parameters:

TOF Calibration Model

The calibrated displacement reading for sensor i is:

$$x_{\text{cal},i} = s_i \cdot (x_{\text{raw},i} - o_i) + b_i \tag{1}$$

where:

- s_i — scale factor (compensates for sensor-to-sensor gain variation)
- o_i — offset (compensates for mounting distance and crosstalk)
- b_i — baseline correction

Typical calibration values:

Sensor	Scale s_i	Offset o_i	Baseline b_i
L5	0.72	23	1
L6	0.67	73	0
L7	0.32	69	1

TOF Sensor Reading — Python Implementation

```python
# From lib/ClassTOFandWebCam.py
def GenerateDisplacementDataFromTOF(self, t1):
    """Read displacement from 6 VL6180X ToF sensors"""
    return np.array([
        self.GenerateTimestamp(t1),
```

```
        self.TOF_sensor[0].range,   # Layer 5
        self.TOF_sensor[1].range,   # Layer 6
        self.TOF_sensor[2].range,   # Layer 7
        self.TOF_sensor[3].range,   # Layer 8
        self.TOF_sensor[4].range,   # Layer 9
        self.TOF_sensor[5].range    # Layer 10
    ])
```

2.3 MCP3008 ADC Pressure Sensor

ADC Sensor Specifications

Pressure sensing uses **Voltage Proportional to Pressure Sensors (VPS)** connected to an analog-to-digital converter.

Parameter	Value
ADC chip	MCP3008 (10-bit, 8 channels)
ADC alternate	MAXIM 11605 (8-bit, 12 channels)
Interface	SPI (MCP3008) / I^2C at `0x65` (MAXIM)
Channels monitored	12 (3 primary: L5, L6, L7)
Measurement unit	mV $\rightarrow$ kPa (with calibration)
Sampling rate	Matched to control loop T_s

The ADC reads the voltage output of each VPS proportional to the pneumatic pressure inside the corresponding silicone chamber.

ADC Pressure Reading — Python Implementation

```python
# From hardware MPC scripts
def GeneratePressureDataFromADC(self, NoOfADC=12):
    """Read 12 ADC channels via MAXIM 11605 ADC"""
    pressure_kpa_array = np.array([
        self.ADCbus.read_byte_data(
            self.ADC_ADDRESS,  # 0x65
            3 << 5 | i << 1 | 1
        ) for i in range(NoOfADC)
    ])
    return pressure_kpa_array
```

2.4 Signal Conditioning — FIR Low-Pass Filter

Raw sensor readings are noisy. A **FIR (Finite Impulse Response)** low-pass filter is applied in real time before feeding measurements to the controller.

FIR Filter Design

The filter is designed using the Kaiser window method:

$$y[n] = \sum_{k=0}^{M-1} b_k \cdot x[n-k] \tag{2}$$

where b_k are the filter coefficients (FIR taps), M is the filter order, and $x[n]$ is the raw sensor measurement at sample n.

Parameters:

Parameter	Value
Number of taps M	10
Normalized cutoff frequency	0.1 (relative to Nyquist)
Window type	Kaiser / Hamming

Real-Time FIR Filtering

```
numtaps_adc = 10
cutoff = 0.1  # Normalized cutoff
b_adc = signal.firwin(numtaps_adc, cutoff)
z_adc = signal.lfilter_zi(b_adc, 1)

# Apply in real-time (per sample)
x_filtered, z_adc = signal.lfilter(
    b_adc, 1, [x_raw_measurement], zi=z_adc
)
```

3 Stage 1 — DTSINDYc Model Training

What is SINDYc?

Sparse Identification of Nonlinear Dynamics with control (SINDYc) is a data-driven method that discovers governing equations from measurement data. Instead of assuming a model structure, SINDYc builds a *library* of candidate functions and uses sparse regression to select only the terms that are active — yielding an interpretable, parsimonious model.

The **discrete-time** variant (DTSINDYc) discovers difference equations:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$$

directly from sampled sensor data, avoiding numerical differentiation entirely.

3.1 Mathematical Formulation

3.1.1 State and Control Vectors

The robot state $\mathbf{x} \in \mathbb{R}^3$ and control input $\mathbf{u} \in \mathbb{R}^3$ are measured at three layers:

State and Control Definitions

Model M1 (TOF-based):

$$\mathbf{x}_k^{\text{TOF}} = \begin{bmatrix} d_{L5}(t_k) \\ d_{L6}(t_k) \\ d_{L7}(t_k) \end{bmatrix} \in \mathbb{R}^3 \quad \text{(displacement in mm)} \tag{3}$$

Model M2 (ADC/VPS-based):

$$\mathbf{x}_k^{\text{ADC}} = \begin{bmatrix} p_{L5}(t_k) \\ p_{L6}(t_k) \\ p_{L7}(t_k) \end{bmatrix} \in \mathbb{R}^3 \quad \text{(pressure in mV)} \tag{4}$$

Control input (shared):

$$\mathbf{u}_k = \begin{bmatrix} v_{L5}(t_k) \\ v_{L6}(t_k) \\ v_{L7}(t_k) \end{bmatrix} \in \mathbb{R}^3 \quad \text{(DAC voltage command)} \tag{5}$$

3.1.2 Polynomial Library Construction

Candidate Function Library Θ

Given state $\mathbf{x} = [x_1, x_2, x_3]^T$ and control $\mathbf{u} = [u_1, u_2, u_3]^T$, the polynomial library of degree $p = 2$ is:

$$\mathbf{\Theta}(\mathbf{x}_k, \mathbf{u}_k) = \Big[\underbrace{1}_{\text{const}}, \underbrace{x_1, x_2, x_3}_{\text{states}}, \underbrace{u_1, u_2, u_3}_{\text{controls}}, \underbrace{x_1^2, x_1x_2, \ldots, u_3^2}_{\text{quadratic}} \Big] \tag{6}$$

For $n = 6$ variables (3 states + 3 controls) and degree $p = 2$, the library contains:

$$\binom{n+p}{p} = \binom{8}{2} = 28 \text{ candidate terms}$$

Library Construction — `sindybase.py`

```python
def polynomial_expansion(self, data, degree=2):
    """
    Expands [x1, x2, x3, u1, u2, u3] into all
    polynomial combinations up to specified degree.
    """
    nfeat = data.shape[-1]  # 6 (states + controls)
    exponents = SINDyBase.get_ordered_poly_exponents(
        nfeat, degree
    )
    result = np.array([
        np.prod(
            [data[:, k] ** e[k] for k in range(nfeat)],
            axis=0
        ) for e in exponents
    ]).T
    return result  # Shape: (T, 28) for degree=2
```

3.1.3 Sparse Regression (STLSQ)

Sequential Thresholded Least Squares (STLSQ)

The DTSINDYc model is:

$$\mathbf{x}_{k+1} = \boldsymbol{\Theta}(\mathbf{x}_k, \mathbf{u}_k)\,\boldsymbol{\Xi} \tag{7}$$

where $\boldsymbol{\Xi} \in \mathbb{R}^{28\times 3}$ is the sparse coefficient matrix.
$\boldsymbol{\Xi}$ is found by solving the sparse regression problem:

$$\min_{\boldsymbol{\Xi}} \|\mathbf{X}_{k+1} - \boldsymbol{\Theta}_k \boldsymbol{\Xi}\|_2^2 \quad \text{subject to} \quad \|\boldsymbol{\Xi}\|_0 \text{ is small} \tag{8}$$

STLSQ Algorithm:

1. Solve the least-squares problem: $\boldsymbol{\Xi}^{(0)} = (\boldsymbol{\Theta}^T\boldsymbol{\Theta})^{-1}\boldsymbol{\Theta}^T\mathbf{X}_{k+1}$
2. For each iteration $j = 1, \ldots, j_{\max}$:
   (a) Set $\Xi_{ij}^{(j)} = 0$ if $|\Xi_{ij}^{(j-1)}| \le \lambda$ (threshold sparsification)
   (b) Re-solve least squares on remaining (non-zero) coefficients
3. Return $\boldsymbol{\Xi}^{(j_{\max})}$

Parameters:

Symbol	Parameter	Value
λ	Sparsification threshold	0.008
α	Regularization (L2 penalty)	0.5
p	Polynomial degree	2
$j_{\max}$	Maximum STLSQ iterations	10

STLSQ Implementation — `sindybase.py`

```python
@staticmethod
def threshold_ls(mtx, _b, lam_bda=1e-3, max_iter=10):
    """
    Sparse regression via sequential thresholding.
    mtx: library Theta (M x 28)
    _b:  next-state data X_{k+1} (M x 3)
    lam_bda: sparsification threshold
    """
    # Step 1: Initial least-squares
    _w = np.linalg.lstsq(mtx, _b, rcond=None)[0]

    # Step 2: Iterative thresholding
    for _ in range(max_iter):
        small_inds = np.abs(_w) <= lam_bda
        _w[small_inds] = 0  # Kill small coefficients

        for ind in range(_b.shape[1]):
            big_inds = ~small_inds[:, ind]
            _w[big_inds, ind] = np.linalg.lstsq(
                mtx[:, big_inds], _b[:, ind],
                rcond=None
            )[0]
    return _w
```

3.1.4 PySINDy Model Fitting

Training a DTSINDYc Model with PySINDy

```python
import pysindy as ps

# Configure optimizer
stlsq_optimizer = ps.STLSQ(
    threshold=0.008,  # lambda
    alpha=0.5         # L2 regularization
)

# Configure polynomial library (degree 2)
poly_library = ps.PolynomialLibrary(
    include_interaction=True,
    interaction_only=False,
    degree=2
)

# Create and fit DTSINDYc model
model = ps.SINDy(
    optimizer=stlsq_optimizer,
    feature_library=poly_library
)

# Discrete-time fitting: x[k+1] = f(x[k], u[k])
model.fit(
    x_train[:-1],        # States at time k
    x_dot=x_train[1:],   # States at time k+1
    u=u_train[:-1]        # Controls at time k
)

model.print()
# Example output for 1-state symmetric:
# (x0)' = 0.812 + 0.985 x0 + 0.189 u0 - 0.003 x0*u0
```

Discovered Model Example — 1-State Symmetric (M2/ADC)

For the 1-state symmetric actuation (all layers move in unison), the DTSINDYc model discovers:

$$x_{k+1} = \underbrace{0.812}_{\text{bias}} + \underbrace{0.985\,x_k}_{\text{autoregressive}} + \underbrace{0.189\,u_k}_{\text{control gain}} - \underbrace{0.003\,x_k u_k}_{\text{nonlinear coupling}} \tag{9}$$

The sparse regression has retained only 4 of 28 candidate terms — a parsimonious model that captures the dominant dynamics.

4 Stage 2 — Reference Trajectory Generation

Peristaltic Wave

Peristalsis is a traveling wave of contraction and relaxation. For the RoSE robot, this is achieved by actuating successive layers with a phase-shifted sinusoidal signal. The wave propagates along the robot body, producing forward locomotion.

4.1 Mathematical Definition

Peristaltic Reference Trajectory (Eq. 14 in Paper)

The reference displacement for layer i at time step t is:

$$h_i(t) = \begin{cases} \epsilon & \text{if } t \leq n_i \cdot \delta \\ \epsilon + \dfrac{\alpha}{2}\left(1 - \cos\left(\dfrac{2\pi(n_i \cdot \delta - t)}{\lambda_d}\right)\right) & \text{if } n_i \cdot \delta < t < n_i \cdot \delta + \lambda_d \\ \epsilon & \text{if } t \geq n_i \cdot \delta + \lambda_d \end{cases} \tag{10}$$

where:

Symbol	Description	Value
α	Wave amplitude	10 (mm or mV)
ϵ	Baseline offset	0
λ_d	Wavelength in samples	$\lambda_{\text{time}}/T_s = 6/0.35 \approx 17$
δ	Propagation delay in samples	$\Delta t_{\text{prop}}/T_s = (15/18.75)/0.35 \approx 2.3$
n_i	Layer index (0, 1, 2)	L5 $\rightarrow$ 0, L6 $\rightarrow$ 1, L7 $\rightarrow$ 2
T_s	Sampling period	0.35 s

4.2 Phase-Shifted Three-Channel Wave

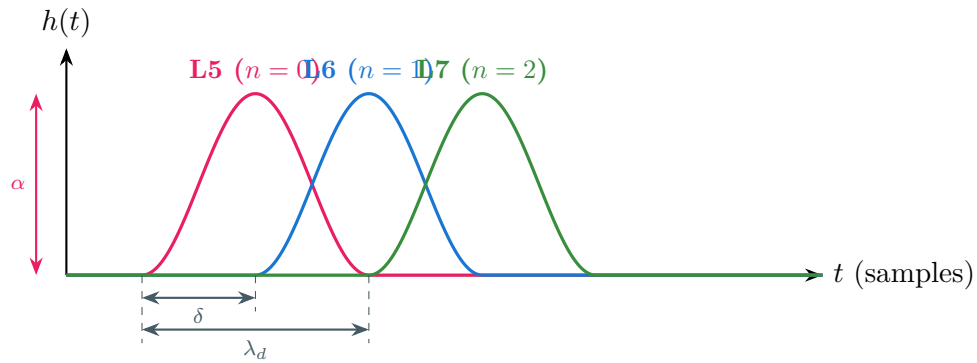


Figure 2: Three-channel peristaltic wave reference trajectory. Each layer is phase-shifted by δ samples, creating a traveling wave.

Trajectory Generation Code

```
Ts = 0.35                # Sampling period (s)
lamb_time = 6            # Wavelength time period (s)
lamb_dis = int(lamb_time / Ts)  # ~17 samples
alpha = 10               # Amplitude
epsilon = 0              # Baseline offset
delay = int((15 / 18.75) / Ts)  # ~2 samples

H = np.zeros((total_samples, 3))

for i, t in enumerate(time_steps):
    for ch in range(3):  # 3 layers
        d_ch = ch * delay  # Phase delay
        if t <= d_ch:
            H[i, ch] = epsilon
```

```
        elif t >= d_ch + lamb_dis:
            H[i, ch] = epsilon
        else:
            H[i, ch] = epsilon + (alpha / 2) * (
                1 - np.cos(2*np.pi*(d_ch - t) / lamb_dis)
            )
```

5 Stage 3 — Model Predictive Control (MPC)

Nonlinear MPC with DTSINDYc Model

The MPC controller uses the identified DTSINDYc model (Eq. 7) as a predictor to optimize future control actions. At each time step k, it solves a constrained optimization problem over a prediction horizon N, applies only the first control action, then re-solves at $k+1$ (receding horizon).

5.1 MPC Cost Function

MPC Optimization Problem

At each time step k, solve:

$$\min_{\mathbf{u}_k,\ldots,\mathbf{u}_{k+N-1}} J = \sum_{j=0}^{N-1} \Big[\underbrace{(\mathbf{x}_{k+j+1} - \mathbf{x}_{\text{ref}})^T \mathbf{Q}\,(\mathbf{x}_{k+j+1} - \mathbf{x}_{\text{ref}})}_{\text{tracking error}} + \underbrace{(\Delta\mathbf{u}_{k+j})^T \mathbf{R}\,\Delta\mathbf{u}_{k+j}}_{\text{control variation}} + \underbrace{\mathbf{u}_{k+j}^T \mathbf{R}_u\,\mathbf{u}_{k+j}}_{\text{control effort}} \Big] \tag{11}$$

subject to:

$$\mathbf{x}_{k+j+1} = \mathbf{\Theta}(\mathbf{x}_{k+j}, \mathbf{u}_{k+j})\,\mathbf{\Xi} \qquad \text{(DTSINDYc model constraint)} \tag{12}$$

$$\mathbf{u}_{\min} \le \mathbf{u}_{k+j} \le \mathbf{u}_{\max} \qquad \text{(actuator bounds)} \tag{13}$$

where $\Delta\mathbf{u}_{k+j} = \mathbf{u}_{k+j} - \mathbf{u}_{k+j-1}$ is the control increment.

5.2 Weight Matrices and Parameters

MPC Tuning Parameters

Symbol	Description	Value
$\mathbf{Q}$	State tracking weight	diag(5, 10, 5)
$\mathbf{R}$	Control variation weight	diag(0.5, 0.5, 0.5)
$\mathbf{R}_u$	Control effort weight	diag(0.5, 0.5, 0.5)
N	Prediction horizon	4 (also tested: 1, 2)
T_s	Sampling time	0.5 s
Solver	Optimization method	SLSQP
tol	Convergence tolerance	0.1

Note: The higher weight on x_2 (layer L6, $Q_{22} = 10$) reflects that the center layer is more critical for peristaltic wave shape accuracy.

5.3 State-Dependent Control Constraints

The actuator bounds are adaptive, depending on the current reference level:

Dynamic Bounds for Channel 1

$$(u_{\min,1},\ u_{\max,1}) = \begin{cases} (40,\ 46) & \text{if } x_{\text{ref},1} \geq 7.6 \\ (32,\ 40) & \text{if } 5.5 \leq x_{\text{ref},1} < 7.6 \\ (24,\ 32) & \text{if } 2.0 \leq x_{\text{ref},1} < 5.5 \\ (9,\ 17) & \text{if } x_{\text{ref},1} < 2.0 \end{cases} \tag{14}$$

Similar adaptive bounds apply to channels 2 and 3. This schedule prevents excessive actuation and ensures the optimization stays near the operating region where the SINDy model is valid.

MPC Objective Function Implementation

```python
def RobotObjectiveFCN(self, u, *args):
    x, Ts, N, xref, u0, Q, R, Ru = args
    xk = x
    u = u.reshape((3, -1))  # (3 channels, N steps)
    J = 0

    for kk in range(N):
        uk = u[:, kk]
        # Predict next state via SINDy model
        xk1 = self.model.predict(
            xk[np.newaxis, :], uk[np.newaxis, :]
        )[0, :]

        # Tracking cost
        e = xk1 - xref
        J += e.T @ Q @ e

        # Control variation cost
        du = uk - (u0 if kk == 0 else u[:, kk-1])
        J += du.T @ R @ du

        # Control effort cost
        J += uk.T @ Ru @ uk

        xk = xk1  # Propagate state
    return J
```

5.4 MPC Block Diagram

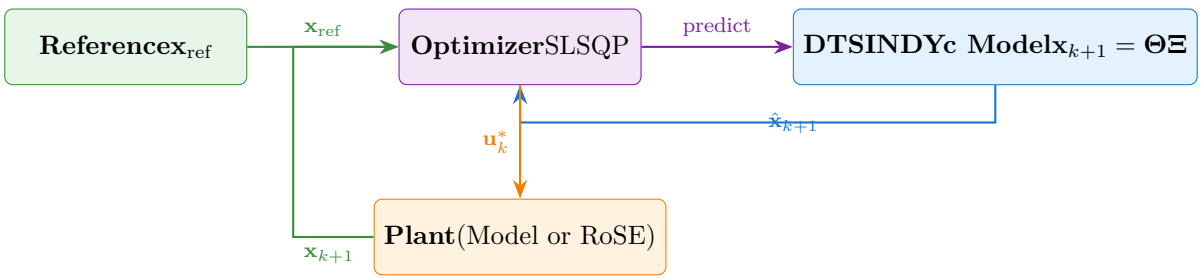
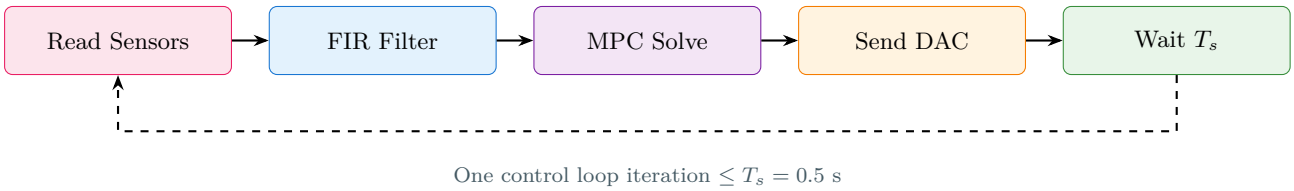


Figure 3: MPC receding-horizon control loop. At each step, the optimizer uses the DTSINDYc model to predict future states and find the optimal control sequence.

6 Stage 4 — Real-Time MPC on Hardware

6.1 Control Loop Architecture



Real-Time MPC Loop on Raspberry Pi

```python
for ct in range(int(Duration / Ts)):
    t_start = time.time()

    # 1. Read sensors
    tof_data = self.GenerateDisplacementDataFromTOF(t1)
    adc_data = self.GeneratePressureDataFromADC(12)

    # 2. Apply FIR filter
    x_filtered, z = Filter_RealTime_apply(
        tof_data[1:4], b_filter, z
    )

    # 3. Get reference for current time step
    xref = xref_trajectory[:, ct]

    # 4. Compute adaptive bounds from xref
    bounds = compute_adaptive_bounds(xref)

    # 5. Solve MPC optimization (SLSQP)
    result = minimize(
        self.RobotObjectiveFCN, u_prev,
        method='SLSQP',
        args=(x_filtered, Ts, N, xref, u0, Q, R, Ru),
        tol=0.1, bounds=bounds
    )
    u_optimal = result.x[:3]  # First step only

    # 6. Send control to 12-layer actuator array
    self.mergeDACadd2DataAndSend(
        u_optimal, 0, BaseLinePress, ScalingFact
    )

    # 7. Wait for next sampling instant
    elapsed = time.time() - t_start
    if elapsed < Ts:
        time.sleep(Ts - elapsed)
```

6.2 3-State to 12-Layer Mapping

The MPC computes 3 control signals (one per monitored layer), but the robot has 12 actuatable layers. The 3 MPC outputs are mapped to all 12 layers using a rotating deque pattern that propagates the peristaltic wave:

Layer Mapping for Peristaltic Actuation

$$\mathbf{u}_{\text{12-layer}}[k] = \mathcal{R}^k\left(\begin{bmatrix} u_1 & u_1 & u_1 & u_1 & u_2 & u_2 & u_2 & u_2 & u_3 & u_3 \\ u_3 & u_3 & & & & & & & & \end{bmatrix}\right) \tag{15}$$

where $\mathcal{R}^k$ denotes a circular shift (rotation) by $k \cdot 3$ positions, creating the traveling wave pattern across the full 12-layer robot body.

7 Stage 5 — Sensor Calibration

Why Calibrate?

Raw sensor readings suffer from:

- **Offset errors** — mounting distance introduces a constant bias
- **Crosstalk** — adjacent TOF sensors interfere (IR reflections)
- **Gain variation** — each VL6180X has slightly different responsivity
- **Convergence time** — sensors need warm-up for stable readings

Calibration must be performed *before* collecting training data or deploying MPC.

7.1 TOF Calibration Procedure

The calibration script (`5_hardware_calibration/`) performs:

1. **Offset calibration**: Place a known target at a reference distance. Record the raw reading and compute $o_i = x_{\text{raw}} - x_{\text{true}}$.
2. **Scale factor**: Measure at multiple known distances. Fit a linear model $x_{\text{true}} = s_i \cdot x_{\text{raw}} + c_i$ via least squares.
3. **Crosstalk compensation**: Fire one sensor at a time and record readings on neighboring sensors. Subtract the crosstalk component.
4. **Convergence time**: Record sensor output after power-on and determine the settling time before readings stabilize (typically 1–2 s).

7.2 ADC Calibration (Full-Scale Pressure)

The MCP3008 ADC is calibrated using a known pressure source:

ADC Full-Scale Calibration

$$p_{\text{cal}} = \frac{V_{\text{ADC}}}{V_{\text{ref}}} \times p_{\text{full-scale}} \tag{16}$$

where $V_{\text{ref}} = 3.3$ V (Raspberry Pi reference voltage) and $p_{\text{full-scale}}$ is determined by the VPS sensor datasheet. The script `Mcp3008_FSP_Cal.py` automates this process.

8 Data Flow and Directory Map

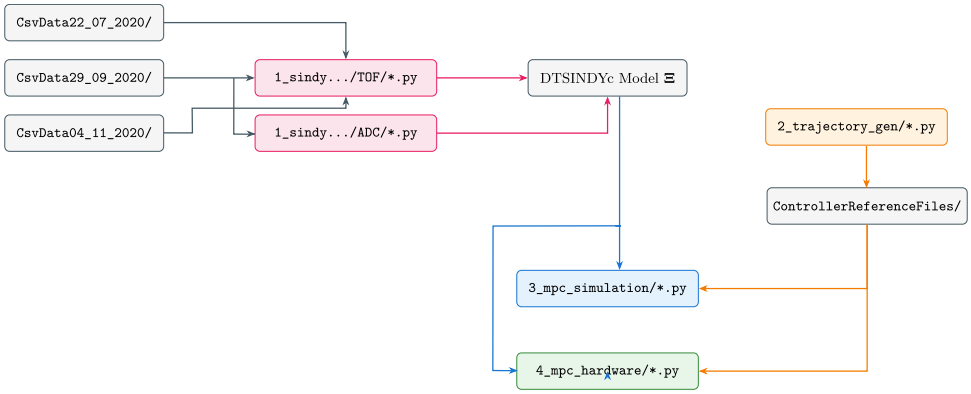


Figure 4: Data flow across the pipeline stages. CSV experimental data feeds into model training (Stage 1), which produces the DTSINDYc model used by both MPC simulation (Stage 3) and hardware deployment (Stage 4).

9 Key Equations Summary

rosebluec

Component	Equation	Eq.
Polynomial Library	$\mathbf{\Theta}(\mathbf{x}, \mathbf{u}) = [1,\ x_1,\ x_2,\ x_3,\ u_1,\ u_2,\ u_3,\ x_1^2,\ x_1x_2,\ \ldots]$	(6)
DTSINDYc Model	$\mathbf{x}_{k+1} = \mathbf{\Theta}(\mathbf{x}_k, \mathbf{u}_k)\,\mathbf{\Xi}$	(7)
STLSQ Objective	$\min_{\mathbf{\Xi}} \lVert \mathbf{X}_{k+1} - \mathbf{\Theta}_k \mathbf{\Xi} \rVert_2^2$ s.t. $\lVert \mathbf{\Xi} \rVert_0$ small	(8)
Peristaltic Wave	$h_i(t) = \epsilon + \frac{\alpha}{2}(1 - \cos(\frac{2\pi(n_i\delta - t)}{\lambda_d}))$	(10)
MPC Cost	$J = \sum_j [(\mathbf{x} - \mathbf{x}_{\text{ref}})^T \mathbf{Q}(\cdot) + \Delta\mathbf{u}^T \mathbf{R} \Delta\mathbf{u} + \mathbf{u}^T \mathbf{R}_u \mathbf{u}]$	(11)
TOF Calibration	$x_{\text{cal}} = s_i(x_{\text{raw}} - o_i) + b_i$	(1)
FIR Filter	$y[n] = \sum_{k=0}^{M-1} b_k \cdot x[n-k]$	(2)

10 Installation and Getting Started

10.1 Environment Setup

Conda Installation (Recommended)

```
# Create environment
conda create -n rose python=3.11 -y
conda activate rose

# Install dependencies
conda install numpy scipy matplotlib pandas scikit-learn -y
pip install pysindy statsmodels python-dateutil
```

10.2 Raspberry Pi Additional Dependencies

RPi Dependencies (Stages 4–5 Only)

```
pip install RPi.GPIO smbus2 spidev
pip install adafruit-blinka adafruit-circuitpython-vl6180x
pip install adafruit-circuitpython-mcp230xx board busio
```

10.3 Running the Pipeline

Step-by-Step Pipeline Execution

```
cd SINDYc_MPC_RoSE_symmetric_peristaltic
conda activate rose

# Stage 1: Train a model (ADC, 1-state symmetric)
python 1_sindy_model_training/ADC/\
  RoSEv2pt0_ADC_SINDYc_using_PySINDY_discrete_\
  states_1_symmetric_07_10.py

# Stage 2: Generate reference trajectory
python 2_trajectory_generation/\
  RoSE_MPC_tof_trajectory_gen.py

# Stage 3: MPC simulation (model as plant)
python 3_mpc_simulation/\
  RoSEv2pt0_ADC_SINDYc_discrete_using_PySINDY_\
  MPC_with_model_as_plant_states_3_peristalsis.py
```

10.4 Repository File Organization

c

Directory	Purpose	Hardware?
1_sindy_model_training/	DTSINDYc model identification	No
2_trajectory_generation/	Peristaltic reference signals	No
3_mpc_simulation/	Offline MPC with model as plant	No
4_mpc_hardware/	Real-time MPC on Raspberry Pi	**Yes**
5_hardware_calibration/	Sensor calibration routines	**Yes**
lib/	Shared library modules	—
csv_data/	Experimental sensor recordings	—
ControllerReferenceFiles/	Reference trajectories (Stage 2 output)	—
PeristalsisData/	Peristaltic waveform parameters	—
TOFCalFiles/	Calibration files (Stage 5 output)	—

11 Appendix: Complete Parameter Reference

Parameter	Symbol	Value	Stage
SINDYc Model Training			
Polynomial degree	p	2	1
STLSQ threshold	λ	0.008	1
L2 regularization	α	0.5	1
Max STLSQ iterations	$j_{\max}$	10	1
Number of states	n_x	3 (or 1)	1
Number of controls	n_u	3 (or 1)	1
Trajectory Generation			
Sampling period	T_s	0.35 s	2
Wavelength (time)	λ_{time}	6 s	2
Wave amplitude	α	10 (mm or mV)	2
Propagation speed	v	18.75 mm/s	2
Layer spacing	Δz	15 mm	2
MPC Controller			
Sampling time	T_s	0.5 s	3, 4
Prediction horizon	N	4	3, 4
State weight	$\mathbf{Q}$	diag(5, 10, 5)	3, 4
Control variation weight	$\mathbf{R}$	diag(0.5, 0.5, 0.5)	3, 4
Control effort weight	$\mathbf{R}_u$	diag(0.5, 0.5, 0.5)	3, 4
Solver	—	SLSQP	3, 4
Tolerance	tol	0.1	3, 4
Hardware			
Control duration	—	290 time steps	4
FIR filter taps	M	10	4
FIR cutoff frequency	f_c	0.1 (normalized)	4
DAC safety cutoff	—	250 (raw units)	4
Robot layers	—	12 (3 monitored)	4